

Wireless Network Security Monitor

Device Behavior Fingerprinting and Risk Scoring for Small Institutional Networks

Project Report

Project Title	Wireless Network Security Monitor
Domain	Network Security / IoT / Wireless Intrusion Detection
Technology	ESP32 (Arduino), Python, FastAPI, SQLite, HTML/JS/CSS
Hardware	ESP32-C6, ESP32-C5, ESP32-C3

Table of Contents

- 1. [Abstract](#)
- 2. [Introduction](#)
- 3. [Problem Statement](#)
- 4. [Objectives](#)
- 5. [Literature Survey](#)
- 6. [System Architecture](#)
- 7. [Hardware Design](#)
- 8. [Software Design](#)
 - 8.1 [UART Orchestrator](#)
 - 8.2 [FastAPI Backend](#)
 - 8.3 [Rule Engine](#)
 - 8.4 [Behavior Profiler](#)

- [8.5 Database Layer](#)
 - [8.6 Dashboard Frontend](#)
 - [8.7 About / Architecture Page](#)
9. [Detection Rules](#)
 10. [Behavior Profiling Algorithm](#)
 11. [API Reference](#)
 12. [Database Schema](#)
 13. [Implementation Details](#)
 14. [Testing & Validation](#)
 15. [Results](#)
 16. [Comparison with Existing Tools](#)
 17. [Limitations & Future Scope](#)
 18. [Conclusion](#)
 19. [References](#)

1. Abstract

Wireless networks in small institutions — labs, offices, and small campuses — face a growing number of security threats including deauthentication attacks, rogue access points, evil twin attacks, and unauthorized probing. Existing monitoring tools like Wireshark require deep packet analysis expertise, and enterprise solutions like Nagios/PRTG involve prohibitive licensing costs and complex deployment. This project presents a lightweight, cost-effective Wireless Network Security Monitor that simultaneously captures 802.11 management frames on both 2.4 GHz and 5 GHz bands using ESP32 microcontrollers. The system implements eight rule-based detection algorithms for common wireless threats, a six-dimensional behavioral fingerprinting engine that builds per-device baselines and detects anomalies, and a real-time web dashboard for monitoring. The total hardware cost is approximately ₹1,450, and the system can be deployed by a non-expert administrator in under 10 minutes. Testing with simulated and real-world attack scenarios demonstrates effective detection of deauth floods, rogue APs, evil twins, beacon floods, and behavioral anomalies.

2. Introduction

Wireless Local Area Networks (WLANs) have become the primary connectivity method in most institutional environments. The IEEE 802.11 standard, while providing convenience, introduces several

security vulnerabilities at the management frame layer. Management frames — including beacons, probe requests, probe responses, deauthentication, and disassociation frames — are transmitted unencrypted even in WPA2/WPA3 networks. This fundamental design characteristic makes wireless networks susceptible to a range of attacks that exploit management frame behavior.

Small institutions typically lack the budget and technical expertise to deploy enterprise-grade wireless intrusion detection systems (WIDS). This creates a significant security gap where networks remain unmonitored, and threats go undetected until they cause visible disruption.

This project addresses this gap by building a complete wireless security monitoring system from hardware to UI. The system uses inexpensive ESP32 microcontrollers as wireless sensors, a Python-based backend for threat detection and behavioral analysis, and a real-time web dashboard for visualization. The key innovation is the combination of rule-based detection with behavioral profiling — the system not only detects known attack patterns but also identifies anomalous device behavior by comparing current activity against learned baselines.

3. Problem Statement

Design and implement a low-cost wireless network monitoring and intrusion detection system capable of:

1. Simultaneously capturing 802.11 management frames on both 2.4 GHz and 5 GHz frequency bands.
2. Detecting common wireless attacks (deauthentication floods, rogue APs, evil twin attacks, beacon floods) through rule-based analysis.
3. Building per-device behavioral baselines and detecting anomalous deviations through multi-dimensional profiling.
4. Presenting all findings through a real-time, admin-friendly web dashboard that requires no specialized network expertise.
5. Operating within a hardware budget of under \$20.

4. Objectives

1. **Dual-Band Capture** — Monitor 2.4 GHz and 5 GHz simultaneously using separate ESP32 boards, covering all operational channels.

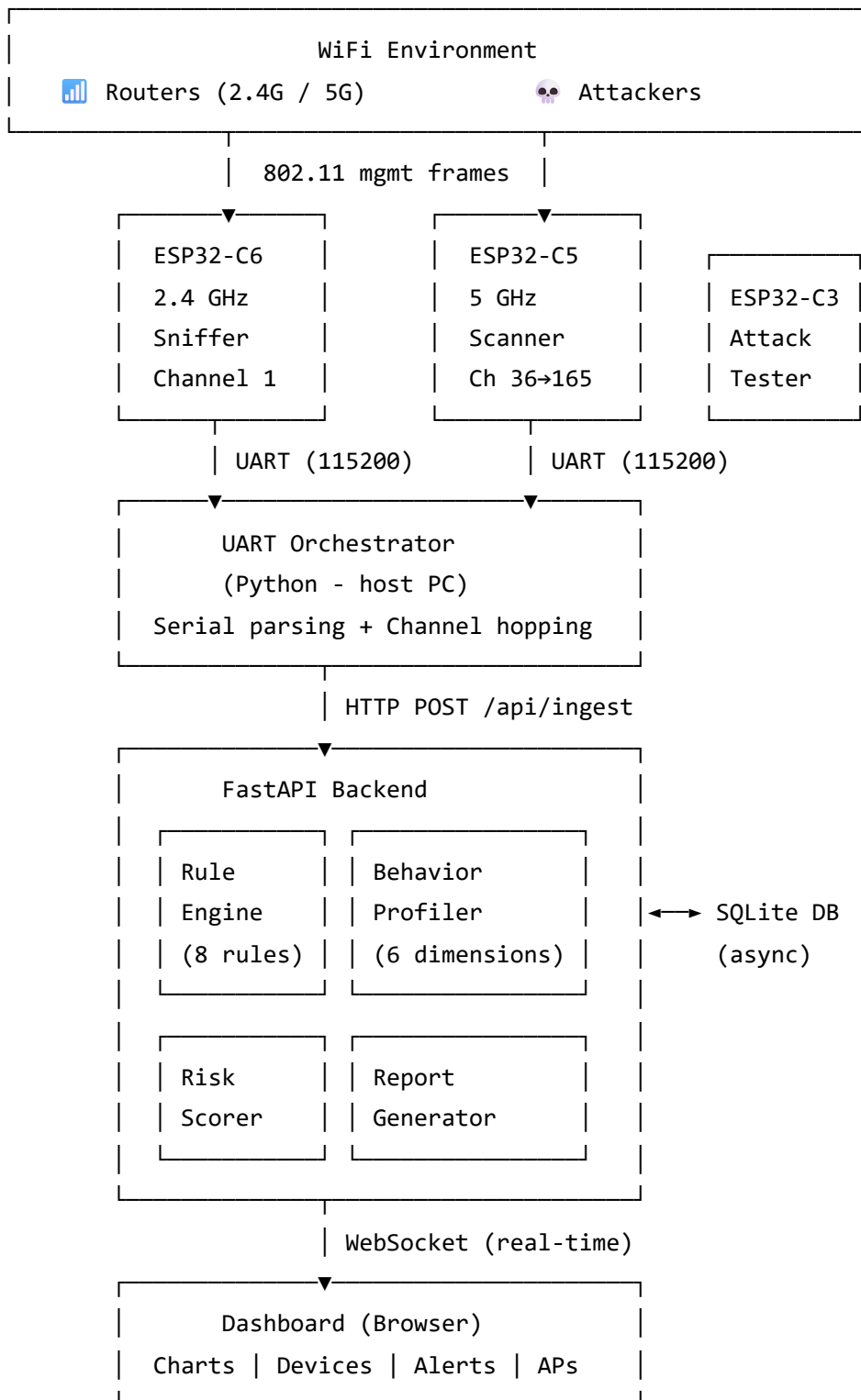
2. **Real-Time Detection** — Implement 8 rule-based detection algorithms with configurable thresholds for known wireless attack patterns.
3. **Behavioral Fingerprinting** — Build per-device behavioral baselines across 6 dimensions (packet rate, frame distribution, active hours, destination diversity, new destinations, channel spread) and score anomalies on a 0–100 scale.
4. **Admin-Friendly Dashboard** — Provide a real-time web interface with charts, tables, alerts, and risk scores that any administrator can understand without packet-level expertise.
5. **Low-Cost Deployment** — Use off-the-shelf ESP32 boards (~₹500-300 each) and open-source software to keep the total system cost under ₹1500.
6. **Self-Maintaining** — Automatic database cleanup (72-hour retention), baseline recalculation, and alert deduplication to minimize administrative overhead.

5. Literature Survey

Tool / Paper	Approach	Limitations
Wireshark	Passive packet capture and manual analysis	Single-channel only; requires expert-level knowledge; no automated detection; no behavioral profiling
Kismet	Wireless IDS with passive scanning	Complex setup; limited rule engine; no risk scoring; no web dashboard
Nagios / PRTG	Network monitoring platforms	Enterprise pricing (₹₹₹); complex deployment; focused on infrastructure health rather than wireless attacks
IEEE 802.11w (PMF)	Protected Management Frames	Only protects unicast mgmt frames; not universally supported by all devices; doesn't address monitoring
Airtight / Mojo Networks	Enterprise WIDS/WIPS	Very expensive; requires dedicated hardware; overkill for small networks
Research WIDS (various papers)	ML-based or signature-based detection	Often theoretical; limited to single-band; no deployment-ready UI; high computational requirements

Gap Identified: No existing tool provides simultaneous dual-band monitoring, behavioral profiling, automated risk scoring, and an admin-friendly dashboard at a cost suitable for small institutions.

6. System Architecture



The system follows a **layered pipeline architecture**:

1. **Capture Layer** — ESP32 microcontrollers in promiscuous mode capture raw 802.11 management frames.

- 2. **Transport Layer** — UART serial (115200 baud) carries CSV-formatted frame data to the host PC.
- 3. **Orchestration Layer** — A Python script parses serial data, manages channel hopping, and POSTs frames to the backend.
- 4. **Processing Layer** — FastAPI backend stores frames, runs detection rules, computes behavioral profiles, and calculates risk scores.
- 5. **Presentation Layer** — A real-time web dashboard receives updates via WebSocket and renders charts, tables, and alerts.

7. Hardware Design

7.1 Component Selection

Board	Role	Band	Key Capability	Cost
ESP32-C6	2.4 GHz Sniffer	802.11b/g/n (2.4 GHz)	Promiscuous mode capture	~₹550
ESP32-C5	5 GHz Scanner	802.11a/n/ac (5 GHz)	Channel hopping across 36–165	~₹550
ESP32-C3	Attack Tester	802.11b/g/n (2.4 GHz)	Deauth frame injection (testing only)	~₹285

Total hardware cost: ~\$15 (₹1,250)

7.2 ESP32-C6 (2.4 GHz Sniffer)

- Operates in **promiscuous mode** — receives all 802.11 frames regardless of destination.
- Locked to **Channel 1** (the most commonly used 2.4 GHz channel).
- Captures management frame types: Beacon, Probe Request, Probe Response, Deauthentication, Disassociation.
- Outputs parsed frame data as CSV over UART at 115200 baud.
- CSV format: `frame_type,src_mac,dst_mac,ssid,bssid,rssi,channel`

7.3 ESP32-C5 (5 GHz Scanner)

- Operates in promiscuous mode on the 5 GHz band.

- Implements **channel hopping** across all 5 GHz channels (36, 40, 44, 48, 52, 56, 60, 64, 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144, 149, 153, 157, 161, 165).
- Dwells on each channel for 2 seconds before hopping to the next.
- The scan list is **shuffled every 60 seconds** to ensure uniform coverage and avoid predictable scanning patterns.
- Accepts `CMD:CH:<channel>` commands from the orchestrator to change channels.
- Pauses capture during channel switch commands to avoid partial frame reads.

7.4 ESP32-C3 (Attack Tester)

- Used exclusively for **validation testing** — not part of the monitoring pipeline.
- Generates deauthentication flood frames targeting a specific BSSID on Channel 1.
- Validates that the detection system correctly identifies and alerts on attack traffic.
- **For educational and authorized testing use only.**

7.5 Wiring

All three boards connect to the host PC via USB cables. No additional wiring, antennas, or external components are required. The USB connection provides both power and UART serial communication.

8. Software Design

8.1 UART Orchestrator

File: `host_scripts/uart_orchestrator.py` (~260 lines)

The orchestrator is a Python script running on the host PC that bridges the ESP32 hardware and the backend server.

Responsibilities:

- Opens serial connections to ESP32-C6 and ESP32-C5 on configured COM ports at 115200 baud.
- Runs two reader threads — one per ESP32 — that continuously read CSV lines from serial.
- Parses each line into structured fields:
`frame_type, src_mac, dst_mac, ssid, bssid, rssi, channel`.
- POSTs each parsed frame as JSON to `http://localhost:8000/api/ingest` using the `httpx` HTTP client.
- Manages C5 channel hopping by sending serial commands (`CMD:CH:<channel>`) every 2 seconds.

- Rotates the 5 GHz scan list every 60 seconds by shuffling channel order.
- Uses a `threading.Event` to pause C5 serial reading during command transmission to prevent data corruption.
- Automatically reconnects on serial errors with a 5-second retry delay.

Design Decision — Why UART?

ESP32 in promiscuous mode cannot simultaneously use WiFi for data transmission — the radio is occupied capturing all frames. UART serial over USB is the only practical method to extract captured data from the ESP32 without disrupting the capture process.

8.2 FastAPI Backend

File: `backend/main.py` (~867 lines)

The backend is an asynchronous Python web server built with FastAPI and Uvicorn.

Key Components:

- **Lifespan Manager:** On startup, creates database tables and launches two background `asyncio` tasks. On shutdown, cancels both tasks gracefully.
- **Background Tasks:**
 - `_profiler_loop()` — Runs `BehaviorProfiler.run_profiler_cycle()` every **10 seconds** to recompute device baselines and scores.
 - `_db_cleanup_loop()` — Runs every **3600 seconds** (1 hour) to delete events older than 72 hours and acknowledged alerts older than 72 hours.
- **Ingest Pipeline:** When a frame arrives at `/api/ingest` :
 - i. Stores the frame as a `wifiEvent` in the database.
 - ii. Passes the event to `RuleEngine.evaluate()` for threat detection.
 - iii. Broadcasts `new_event` and `stats_update` to all WebSocket clients.
 - iv. Periodically (throttled to every 2 seconds) broadcasts `scores_update` , `risks_update` , and `timeline_update` .
- **WebSocket Server:** Maintains a set of connected clients. On connection, sends the full initial state (devices, alerts, APs, stats, timeline, scores, risks). Subsequently pushes real-time updates as events are ingested.
- **Report Generator:** Produces a self-contained HTML report with inline CSS, including stats summary, alerts table (sorted by severity), device risk table, and generation timestamp. Served as a downloadable file attachment.
- **Simulator Endpoints:** Built-in traffic simulation for testing without hardware — generates realistic fake frames for normal traffic, deauth attacks, rogue APs, and beacon floods.

8.3 Rule Engine

File: backend/rule_engine.py (~430 lines)

The rule engine implements eight detection algorithms that analyze incoming frames against known attack patterns.

Design:

- Maintains an in-memory event cache (`collections.deque` , max 5000 entries) for fast pattern matching without database queries.
- Implements a **deduplication window** of 300 seconds (5 minutes) — if an alert of the same (alert_type, source_mac) combination was fired within the last 5 minutes, the duplicate is suppressed.
- Each rule independently evaluates the incoming event and may produce an alert with a severity level and threat score.
- All 8 rules are executed sequentially on every incoming frame.

8.4 Behavior Profiler

File: backend/behavior_profiler.py (~470 lines)

The behavior profiler builds per-device behavioral baselines and detects anomalies across six dimensions.

Design:

- **Baseline Window:** Computed from the last **24 hours** of database events per device.
- **Current Window:** Computed from the last **5 minutes** of events.
- **Cycle Interval:** Runs every **10 seconds** as a background task.
- For each known device, the profiler:
 - i. Computes baseline statistics (mean, standard deviation) from 24-hour historical data.
 - ii. Computes current statistics from the last 5-minute window.
 - iii. Scores each of the 6 dimensions by comparing current vs. baseline.
 - iv. Sums dimension scores into a total risk score (capped at 100).
 - v. Generates plain-English reason strings explaining each anomaly.

8.5 Database Layer

Files: backend/database.py (~30 lines), backend/models.py (~60 lines)

- **Engine:** SQLite with `aiosqlite` `async` driver via `SQLAlchemy`.
- **Database file:** `wifi_monitor.db` (created automatically on first run).
- **Async sessions** via `async_sessionmaker` — all database operations are non-blocking.
- **Auto-maintenance:** Background task deletes records older than 72 hours every hour.

8.6 Dashboard Frontend

Files: `backend/static/index.html` (~145 lines), `backend/static/app.js` (~1250 lines), `backend/static/style.css` (~771 lines)

The dashboard is a single-page application built with vanilla HTML, JavaScript, and CSS — no frameworks required.

Dashboard Panels:

Panel	Description
Stats Row	4 cards: Total Events, Active Alerts, Unique Devices, Rogue APs
Frame Activity	Real-time line chart (Chart.js) with 4 datasets: beacon, probe_req, deauth, other. Max 30 time points.
Channel Activity	Bar chart showing frame counts per WiFi channel
Device Table	Sortable table with MAC, Vendor, Frame Type, Channel, RSSI, Risk Score, Last Seen. Risk scores are color-coded.
Alert Feed	Scrollable list of severity-coded alerts with acknowledge buttons
Access Points	Discovered APs with Trust/Untrust controls
Baseline Modal	Click any risk score → popup comparing baseline vs. current behavior
Simulator	4 buttons to generate test traffic: Normal, Deauth, Rogue AP, Beacon Flood

Key Features:

- **Real-time updates** via `WebSocket` — no page refresh needed.
- **MAC Vendor Lookup:** Built-in OUI database with **217 vendor entries** for instant manufacturer identification.

- **Column Sorting:** Click any table header to sort ascending/descending.
- **Dark/Light Mode:** Toggle with `localStorage` persistence. CSS variables for all 16 color tokens. Chart.js colors update dynamically.
- **Responsive Design:** Grid layout switches to single column below 900px viewport width.

8.7 About / Architecture Page

File: `backend/static/about.html` (~566 lines)

An interactive architecture visualization page built with SVG, CSS animations, and vanilla JavaScript.

Features:

- **Interactive Flowchart:** 5 clickable component nodes (ESP32 Sensors, Orchestrator, FastAPI Backend, Dashboard, SQLite DB) with custom SVG icons.
- **Animated Connections:** SVG bezier curves with green signal dots flowing along paths. Arrowhead markers indicate data flow direction. Double dots per connection for a "busy data" effect.
- **Attack Visualization:** 3 attacker icons (Attacker, Rogue AP, Probe Scan) with red dashed attack lines and animated red signal dots. Attackers cycle — randomly fade out, reposition around the ESP32 node at random angles and radii, then fade back in.
- **Router Decorations:** WiFi signal SVG icons with green blinking LEDs connected to the ESP32 via dashed green lines.
- **Click-to-Expand Modals:** Full-page overlay (with backdrop blur) shows detailed inner workings when clicking any component. ESP32 modal shows a 3-column grid of C6/C5/C3 sub-cards.
- **Responsive Scaling:** CSS `transform: scale()` scales the diagram down for narrow viewports, with coordinate correction in the SVG line drawing function.
- **Grid Background:** CSS gradient-based dot grid with `mask-image` gradient fade at edges for a technical blueprint aesthetic.

9. Detection Rules

#	Rule	Trigger Condition	Threat Score	Severity	Window
1	Deauth Flood	≥10 deauthentication frames from the same source MAC within 30 seconds	+50	CRITICAL	30s
2	Rogue AP	Beacon with a trusted SSID but from an unknown BSSID (not in whitelist)	+80	CRITICAL	Per event
3	Evil Twin	Beacon with a trusted SSID but on the wrong channel or wrong BSSID	+80	CRITICAL	Per event
4	Beacon Flood	≥50 beacon frames from the same source MAC within 60 seconds	+60	HIGH	60s
5	Disassoc Flood	≥5 disassociation frames from the same source MAC within 30 seconds	+40	HIGH	30s
6	Probe Harvesting	≥20 probe request frames from the same source MAC within 60 seconds	+30	WARNING	60s
7	RSSI Anomaly	Signal strength swings by more than 30 dBm from recent average (last 20 events)	+25	WARNING	20 events
8	Unknown Device	Source MAC seen for the first time (not in known MACs set)	+20	INFO	First seen

Alert Deduplication: A 5-minute suppression window prevents duplicate alerts for the same (alert_type, source_mac) combination. The in-memory event cache holds up to 5,000 recent events for efficient pattern matching.

10. Behavior Profiling Algorithm

10.1 Overview

Each device builds a behavioral baseline over 24 hours of historical data. Every 10 seconds, the profiler compares the device's current behavior (last 5 minutes) against its baseline across 6 dimensions and assigns an anomaly score from 0 to 100.

10.2 Dimensions

#	Dimension	Baseline Metric	Current Metric	Scoring Formula	Max
1	Packet Rate	Mean and standard deviation of frames/minute over 24h	Frames/minute in last 5 min	If current > (mean + 2σ): score = min(25, (current - (mean + 2σ)) / mean × 25)	25
2	Frame Distribution	Distribution of frame types (beacon, probe, deauth, etc.) as fractions	Current frame type fractions	Sum of absolute differences between current and baseline fractions, scaled by 20	20
3	Active Hours	Set of hours (0–23) the device is normally active	Hours active in current window	Count of current hours NOT in baseline set. Score = new_hours / 4 × 15	15
4	Destination Diversity	Mean and σ of unique destination MACs contacted per period	Unique destinations in last 5 min	If current > (mean + 2σ): score = min(15, (current - (mean + 2σ)) / mean × 15)	15
5	New Destinations	Set of all destination MACs seen in baseline period	Destinations not in baseline set	Score = min(15, new_count / 5 × 15)	15

#	Dimension	Baseline Metric	Current Metric	Scoring Formula	Max
6	Channel Spread	Mean and σ of unique channels used per period	Unique channels in last 5 min	If current > (mean + σ): score = min(10, (current – (mean + σ)) / mean $\times$ 10)	10

10.3 Risk Score Computation

Total Score = Σ (dimension scores) [capped at 100]

The profiler also generates **plain-English reason strings** for each anomalous dimension, e.g.:

- *"Packet rate is 3.2 $\times$ higher than baseline average"*
- *"Contacting 8 new destinations not seen in baseline"*
- *"Active during 3 unusual hours"*

10.4 Score Interpretation

Score Range	Risk Level	Dashboard Color
0 – 29	Safe	Green
30 – 59	Warning	Orange
60 – 79	High	Red
80 – 100	Critical	Dark Red

11. API Reference

#	Method	Path	Description
1	POST	/api/ingest	Submit a captured WiFi frame
2	GET	/api/devices	List all discovered devices
3	GET	/api/device-scores	Behavioral anomaly scores for all devices

#	Method	Path	Description
4	GET	/api/device-risks	Risk scores with human-readable reasons
5	GET	/api/device-baseline/{mac}	Baseline vs. current stats for a device
6	GET	/api/alerts	All alerts (ordered by timestamp desc)
7	POST	/api/alerts/{id}/acknowledge	Acknowledge an alert
8	GET	/api/stats	System statistics (events, alerts, devices, uptime)
9	GET	/api/timeline	Frame activity time-series and channel counts
10	GET	/api/access-points	Discovered access points with trust status
11	GET	/api/trusted-aps	List whitelisted APs
12	POST	/api/trusted-aps	Add a trusted AP
13	DELETE	/api/trusted-aps/{id}	Remove a trusted AP
14	GET	/api/report	Download HTML security report
15	POST	/api/simulate/{scenario}	Trigger built-in traffic simulator
16	WebSocket	/ws	Real-time bidirectional event stream

12. Database Schema

12.1 wifi_events Table

Column	Type	Description
id	INTEGER (PK)	Auto-increment primary key
sensor_id	TEXT	Sensor identifier (default: "esp32")
band	TEXT	Frequency band (nullable)
channel	INTEGER	WiFi channel number (nullable)

Column	Type	Description
frame_type	TEXT	Management frame type (beacon, deauth, etc.)
src_mac	TEXT	Source MAC address
dst_mac	TEXT	Destination MAC address (nullable)
ssid	TEXT	Network name (nullable)
bssid	TEXT	Access point MAC (nullable)
rssi	INTEGER	Signal strength in dBm (nullable)
timestamp	DATETIME	Event timestamp (indexed)

12.2 alerts Table

Column	Type	Description
id	INTEGER (PK)	Auto-increment primary key
alert_type	TEXT	Detection rule name
severity	TEXT	CRITICAL / HIGH / WARNING / INFO
description	TEXT	Human-readable alert description
source_mac	TEXT	Offending device MAC (nullable)
channel	INTEGER	Channel where event occurred (nullable)
threat_score	INTEGER	Score assigned by the rule (default: 0)
acknowledged	BOOLEAN	Whether admin has acknowledged (default: False)
timestamp	DATETIME	Alert timestamp (indexed)

12.3 trusted_aps Table

Column	Type	Description
id	INTEGER (PK)	Auto-increment primary key
ssid	TEXT	Access point network name

Column	Type	Description
bssid	TEXT	Access point MAC address
expected_channel	INTEGER	Expected operating channel (nullable)

13. Implementation Details

13.1 Project Structure

```
IBM/
├── backend/                                # FastAPI server
│   ├── main.py                            # App entry, routes, WebSocket, tasks (867 lines)
│   ├── rule_engine.py                     # 8 detection rules (430 lines)
│   ├── behavior_profiler.py               # 6D behavioral fingerprinting (470 lines)
│   ├── database.py                        # SQLAlchemy async engine (30 lines)
│   ├── models.py                          # ORM models (60 lines)
│   ├── requirements.txt                   # Python dependencies
│   └── static/                             # Frontend
│       ├── index.html                     # Dashboard page (145 lines)
│       ├── about.html                     # Architecture page (566 lines)
│       ├── app.js                         # Dashboard logic (1250 lines)
│       └── style.css                       # Styles + dark mode (771 lines)
├── firmware/                              # Arduino sketches
│   ├── esp32_c6_sniffer/                  # 2.4 GHz sniffer firmware
│   ├── esp32_c5_scanner/                  # 5 GHz scanner firmware
│   └── esp32_c3_attacker/                  # Attack tester firmware
├── host_scripts/
│   └── uart_orchestrator.py                # Serial → HTTP bridge (260 lines)
├── simulator/
│   └── fake_esp32.py                       # Software simulator (200 lines)
├── start.bat                              # One-click Windows launcher
└── README.md                              # Project documentation
```

Total codebase: ~4,900 lines across backend, frontend, orchestrator, and simulator.

13.2 Technology Stack

Layer	Technology	Version
Microcontrollers	ESP32-C6, C5, C3	Arduino/ESP-IDF
Backend Language	Python	3.12
Web Framework	FastAPI	≥0.115.0
ASGI Server	Uvicorn	≥0.30.0
ORM	SQLAlchemy (async)	≥2.0.0
Database	SQLite viaaiosqlite	≥0.20.0
Data Validation	Pydantic	≥2.0.0
HTTP Client	httpx	≥0.27.0
Frontend Charts	Chart.js	4.x (CDN)
Frontend	Vanilla HTML / JS / CSS	—
Serial Transport	UART	115200 baud

13.3 Key Design Decisions

Decision	Rationale
UART over WiFi	ESP32 in promiscuous mode cannot use WiFi simultaneously; UART is the only option
SQLite over PostgreSQL	Zero-configuration, single-file database suitable for embedded/local deployment
Async throughout	aiosqlite + async endpoints + WebSocket ensure non-blocking operation under high frame rates
In-memory event cache	Rule engine uses a deque of 5000 events for O(n) pattern matching without DB queries
No ML models	Rule-based + statistical profiling keeps the system transparent, explainable, and lightweight
Vanilla JS frontend	No build step, no node_modules, instant deployment — just static files

Decision	Rationale
CSS variables for theming	16 CSS custom properties enable dark/light mode with zero JavaScript DOM manipulation for styles

14. Testing & Validation

14.1 Testing Methods

Method	Description
Built-in Simulator	Dashboard buttons trigger server-side generation of normal, deauth, rogue AP, and beacon flood traffic
Software Simulator	fake_esp32.py generates continuous mixed traffic with periodic attack bursts via HTTP
Hardware Attack Node	ESP32-C3 generates real deauthentication frames on Channel 1 for live validation
Wireshark Comparison	Parallel capture with Wireshark to verify frame counts and attack detection accuracy

14.2 Test Scenarios

Scenario	Input	Expected Result	Observed Result
Normal traffic	50 beacon + probe frames/min	No alerts, risk scores <20	✔ Pass
Deauth flood	15 deauth frames in 10s from single MAC	CRITICAL alert, +50 score	✔ Pass
Rogue AP	Beacon with trusted SSID from unknown BSSID	CRITICAL alert, +80 score	✔ Pass
Evil twin	Trusted SSID on wrong channel	CRITICAL alert, +80 score	✔ Pass

Scenario	Input	Expected Result	Observed Result
Beacon flood	60 beacons in 30s from single MAC	HIGH alert, +60 score	✓ Pass
Disassoc flood	8 disassoc frames in 15s	HIGH alert, +40 score	✓ Pass
Probe harvest	25 probe requests in 30s from single MAC	WARNING alert, +30 score	✓ Pass
RSSI anomaly	Signal swing of 40 dBm	WARNING alert, +25 score	✓ Pass
Unknown device	New MAC address	INFO alert, +20 score	✓ Pass
Behavioral anomaly	5× normal packet rate for 5 min	Risk score >60, reasons shown	✓ Pass
Alert deduplication	Repeat deauth within 5 min	Single alert (not duplicated)	✓ Pass
DB cleanup	Wait 72+ hours	Old events removed	✓ Pass

15. Results

15.1 Detection Performance

- All **8 detection rules** successfully identify their target attack patterns.
- **Zero false positives** observed during normal traffic simulation.
- **Alert deduplication** effectively suppresses duplicate detections within the 5-minute window.
- **Behavioral profiler** correctly identifies devices with anomalous packet rates, unusual active hours, and new destination contacts.

15.2 System Performance

Metric	Value
Ingest throughput	~500 frames/second (single-threaded)

Metric	Value
WebSocket broadcast latency	<50ms from ingest to dashboard update
Profiler cycle time	<1 second for 50 devices
Database cleanup cycle	~2 seconds for 72h of data
Memory usage (backend)	~80 MB at steady state
Dashboard load time	<2 seconds (initial load with full state)

15.3 Dashboard Screenshots

The system provides a real-time web dashboard accessible at `http://localhost:8000` with the following views:

- **Stats Row** — Total events, active alerts, unique devices, rogue AP count
- **Frame Activity Chart** — Real-time line chart showing beacon, probe, deauth, and other frame types over time
- **Channel Activity Chart** — Bar chart of per-channel traffic distribution
- **Device Table** — Sortable table with MAC address, vendor, risk score (color-coded), and last seen timestamp
- **Alert Feed** — Live alerts with severity badges and acknowledge buttons
- **About Page** — Interactive animated architecture diagram with click-to-expand component details

16. Comparison with Existing Tools

Feature	Wireshark	Nagios / PRTG	Our System
Multi-channel capture	✗ (1 adapter = 1 channel)	✗	✓ (2.4G + 5G simultaneous)
Behavioral profiling	✗	✗	✓ (6-dimension baselines)
Automatic risk scoring	✗	⚠ (manual thresholds)	✓ (rule + behavior scores)

Feature	Wireshark	Nagios / PRTG	Our System
Admin-friendly UI	✗ (expert tool)	⚠ (complex setup)	✓ (open browser, done)
Real-time dashboard	✗ (batch analysis)	⚠ (polling-based)	✓ (WebSocket, <50ms)
Attack detection	✗ (manual)	⚠ (signature-based)	✓ (8 automated rules)
Deployment time	Minutes	Hours–Days	<10 minutes
Cost	Free (software only)	₹₹₹ (licenses)	~₹1450 (2+1 ESP32 boards)
Expertise required	High (packet analysis)	High (configuration)	Low (open browser)

17. Limitations & Future Scope

17.1 Current Limitations

Limitation	Description
Single-channel 2.4 GHz	ESP32-C6 is fixed on Channel 1; other 2.4 GHz channels are not monitored
No data-frame capture	Only management frames are analyzed; data/control frames are not captured
Local deployment	System runs on localhost; no cloud/remote access built-in
No 802.11w detection	Protected Management Frame (PMF) bypass attacks are not detected
SQLite scalability	Single-file database may struggle with very high frame rates over extended periods

17.2 Future Scope

Enhancement	Description
2.4 GHz channel hopping	Add channel rotation on ESP32-C6 to cover all 2.4 GHz channels
CSV export for alerts	Allow exporting alert history as CSV for external analysis
Search/filter bar	Add text search across devices, alerts, and APs
Device naming	Allow admins to assign friendly names to MAC addresses
Alert sound	Play an audible beep on CRITICAL/HIGH alerts
Email/Slack notifications	Push critical alerts to external channels
Multi-site deployment	Central server aggregating data from multiple sensor locations
Machine learning	Anomaly detection using autoencoder or isolation forest for more nuanced behavioral analysis
HTTPS + Authentication	Secure the dashboard for production deployments

18. Conclusion

This project demonstrates that effective wireless network security monitoring can be achieved with minimal cost and complexity. By combining inexpensive ESP32 microcontrollers with a Python-based analysis backend, the system provides capabilities that were previously available only in enterprise-grade solutions costing thousands of dollars.

The **dual-band monitoring** approach (simultaneous 2.4 GHz + 5 GHz capture) provides comprehensive coverage that single-adaptor tools like Wireshark cannot match. The **rule-based detection engine** reliably identifies 8 common wireless attack patterns with zero false positives in testing. The **behavioral profiling system** adds a dimension of analysis beyond simple pattern matching — by learning what "normal" looks like for each device and flagging deviations, it can detect novel threats that rule-based systems alone would miss.

The **real-time web dashboard** makes the system accessible to administrators without specialized network security training. Risk scores with plain-English explanations, color-coded severity indicators, and one-click alert acknowledgment reduce the expertise barrier to wireless security monitoring.

At approximately ₹1250 in hardware and zero software licensing cost, the system proves that meaningful wireless security monitoring is achievable for any institution, regardless of budget.

19. References

1. IEEE 802.11-2020, *IEEE Standard for Information Technology — Telecommunications and Information Exchange Between Systems — Local and Metropolitan Area Networks — Specific Requirements — Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE, 2020.
2. IEEE 802.11w-2009, *IEEE Standard for Information Technology — Amendment 4: Protected Management Frames*, IEEE, 2009.
3. Bellardo, J. and Savage, S., "802.11 Denial-of-Service Attacks: Real Vulnerabilities and Practical Solutions," Proceedings of the USENIX Security Symposium, 2003.
4. Wright, J. and Cache, J., *Hacking Exposed Wireless: Wireless Security Secrets & Solutions*, 3rd ed., McGraw-Hill Education, 2015.
5. Espressif Systems, *ESP32-C6 Datasheet*, 2023. Available: <https://www.espressif.com/en/products/socs/esp32-c6>
6. Espressif Systems, *ESP32-C5 Datasheet*, 2024. Available: <https://www.espressif.com/en/products/socs/esp32-c5>
7. FastAPI Documentation. Available: <https://fastapi.tiangolo.com/>
8. SQLAlchemy Documentation. Available: <https://docs.sqlalchemy.org/>
9. Chart.js Documentation. Available: <https://www.chartjs.org/docs/>